Advanced RAG System: Query Rewriting, Hybrid Retrieval, and Citation Generation

By Insharah Irfan Nazir

1. Executive Summary 1
2. Introduction 1
3. System Architecture: End-to-End Processing Flow 1
4. Implemented Components 3
    4.1 Query Rewriting 3
    4.2 Multi-Query Retrieval 3
    4.3 Hybrid Search (BM25 + Vector) 4
    4.4 Graph-Augmented Retrieval 4
    4.5 Context Reranking 5
    4.6 Citation Generation 5
    4.7 End-to-End API 6
5. Evaluation Methodology 6
6. Retrieval Evaluation Results 6
    6.1 Overall 7
    6.2 By Language 8
    6.3 Language Performance Gap: Root Cause 10
7. Citation Generation Quality: Qualitative Assessment 11
8. Findings and Limitations 12
    8.1 Reranker Miscalibration on Abstract Philosophical Text 12
    8.2 Latency and Free-Tier LLM Variance 12
    8.3 Response Validation 13
    8.4 Chunk-Boundary Limitation (Multi-Fact Queries) 13
    8.5 Non-Determinism 14
9. Graph Search: Development Notes 14
10. Conclusion 15

1. Executive Summary

This report documents the third phase of a multi-stage RAG pipeline, building directly on the chunking benchmark (Phase 1) and the retrieval benchmark (Phase 2). Phase 2 established semantic chunking + BGE-M3 embeddings + FAISS as the optimal retrieval foundation (0.556 recall@1, 0.867 recall@10, 0.661 MRR on the 45-query multilingual eval set). This phase adds six retrieval and generation components on top of that foundation: query rewriting, multi-query retrieval, hybrid search (BM25 + vector), graph-augmented retrieval, context reranking, and citation-grounded answer generation, exposed through an end-to-end FastAPI service.

Retrieval evaluation on the full 45-query set (English, Korean, Urdu) shows the full pipeline achieves **0.556 recall@1, 0.911 recall@10, and 0.673 MRR** — an improvement over the pre-graph pipeline's 0.533 recall@1, 0.889 recall@10, and 0.652 MRR. Graph-augmented retrieval was developed and validated in two stages: an initial standalone evaluation showed no net-new retrieval value when queried on raw user input alone (Section 9), which motivated extending graph search to run across the rewritten query and all multi-query variants rather than the raw query only. Under that integration, the full pipeline showed a measurable, if modest, improvement across all three languages, concentrated most heavily in English.

2. Introduction

A production RAG system needs more than a single embedding-and-search step. Real user queries are often vague, ambiguous, or reference entities implicitly; a single retrieval pass can miss relevant chunks that use different vocabulary than the query; named entities that appear verbatim in both query and source text can be under-weighted by semantic and lexical methods alike; and generated answers need to be traceable back to source material rather than presented as unattributed fact. This phase addresses each of these gaps:

- **Query rewriting** cleans up vague or colloquial queries before retrieval.
- **Multi-query retrieval** casts a wider net by generating phrasing variants, reducing the risk of a single bad rewrite tanking retrieval.
- **Hybrid search** combines BM25 (lexical, exact-term matching) with vector search (semantic similarity), since the two methods fail in complementary ways.
- **Graph-augmented retrieval** adds a third candidate source based on named-entity co-occurrence, catching cases where a query and its answer share proper nouns without sharing enough vocabulary or embedding similarity for hybrid search alone to surface the connection.
- **Context reranking** uses a cross-encoder to jointly score query-chunk pairs, correcting ranking errors that neither BM25, vector search, nor graph search can catch in isolation.
- **Citation generation** grounds every generated answer in specific retrieved chunks and withholds a confident answer when retrieval confidence is low.

3. System Architecture: End-to-End Processing Flow

The system processes user queries through a sequential pipeline of seven stages, each building on the output of the previous stage:

1. Query Rewriting: The raw user query is passed to an LLM, which reformulates it into a clearer, more search-friendly phrase while preserving the original intent. For example, "whats camus say about death and stuff" becomes "What does Camus say about death and the absurd?" This step addresses the common RAG failure mode where vague or colloquial queries fail to retrieve relevant content.
2. Multi-Query Generation: The rewritten query is expanded into 3–4 alternative phrasing variants via a second LLM call. Each variant explores different angles or resolves ambiguous references differently, ensuring that a single lossy rewrite cannot tank the entire retrieval process. Critically, generation is conditioned on both the original query and the rewritten query to prevent error propagation.
3. Parallel Hybrid Retrieval: Each query variant is independently run through two complementary retrieval systems:
    - Vector Search (FAISS): Uses BGE-M3 embeddings to find semantically similar chunks, strong on meaning-based matching but weak on exact terms.
    - BM25 Search: Uses lexical term-matching to find chunks with exact vocabulary overlap, strong on specific names and terminology but blind to semantics.
    - Both searches run in parallel for each variant, producing separate ranked candidate lists.
4. Graph-Augmented Retrieval: the raw query, the rewritten query, and every multi-query variant are each checked against a pre-built entity co-occurrence graph. Chunks whose entities match query entities are added to the candidate pool as a third retrieval source, scored on the same scale as the RRF-fused hybrid results (Section 4.4).
5. RRF Fusion and Filtering: All candidate lists from both retrieval methods and all query variants are merged using Reciprocal Rank Fusion (RRF, k=60), which combines rankings by position rather than raw score (since BM25 scores and cosine similarities are on incomparable scales). After fusion, candidates scoring below 50% of the top fused score are dropped—a safeguard against weakly-ranked candidates that might otherwise receive inflated reranker scores.
6. Cross-Encoder Reranking: The remaining candidates are re-scored by BAAI/bge-reranker-base, a cross-encoder that reads each query-chunk pair jointly rather than comparing independent representations. This is more accurate than either lexical overlap or embedding similarity, but computationally expensive—hence it runs only on the filtered candidate pool, not the full corpus. Inputs are truncated to 256 tokens for latency control.
7. Confidence Gating and Context Assembly: The top reranked chunks are assembled into context. If the highest reranker score falls below 0.5, the system halts here and returns a hedging response ("the corpus doesn't clearly address this question...") rather than attempting an answer from weak grounding—a direct defense against hallucination.
8. Citation-Grounded Generation: The LLM generates an answer using only the retrieved context, with explicit instructions to:
    - Cite every claim with bracketed chunk indices ([1], [2], etc.)
    - Decline to state anything not directly supported
    - Distinguish between fictional content and factual claims when ambiguous

9. The returned source list includes only chunks actually cited in the answer (parsed via regex), giving callers an accurate picture of grounding, rather than every chunk passed into the context window.

The full pipeline is exposed through a FastAPI service with a single /query endpoint, and all models and indices are loaded once at startup rather than per-request to minimize latency.

The system reuses Phase 1's chunked corpus (document-ingestion-pipeline/processed_documents/) and Phase 2's precomputed BGE-M3 embeddings and FAISS adapter directly, rather than reprocessing the corpus — the two earlier phases and this one share a corpus of 10,456 chunks across 63 documents (English, Korean, Urdu). All models and indices (FAISS, BM25, embedding model, reranker) are loaded once at API startup rather than per-request. The entity graph (Section 4.4) is built once, offline, from the same corpus, and loaded once at API startup alongside the other indices.

LLM calls (query rewriting, multi-query generation, citation generation) run through OpenRouter's free-tier model router rather than a paid API, which materially shaped both the system's design (retry/validation logic) and its measured latency (Section 8).

4. Implemented Components

4.1 Query Rewriting

Raw user queries are often too vague, colloquial, or ambiguous to retrieve well against. A single LLM call rewrites the query into a clearer search phrase while enforcing several constraints developed through iteration:

- Preserve intent exactly; do not answer the question.
- Do not invent titles, names, or facts not stated or clearly implied by the query.
- Do not substitute specific nouns with broader or different terms (e.g. "job" → "role" was an observed failure mode that silently changed query semantics).
- Keep enough detail for the query to remain useful for semantic search — an earlier iteration over-shortened rewrites into bare keyword fragments, which is worse for retrieval than a natural, specific phrase.

Example: "whats camus say about death and stuff" → "What does Camus say about death and the absurd?"

4.2 Multi-Query Retrieval

A single rewritten query is expanded into several phrasing variants (typically 3–4, including the rewrite itself) via a second LLM call. Variants explore different angles or possible resolutions of ambiguous references, without asserting unconfirmed facts as certain. Critically, variant generation is conditioned on **both the raw query and the rewritten query**, not the rewrite alone — an earlier design that only saw the rewritten query allowed a single lossy rewrite to propagate uncorrected through all variants (e.g. a

dropped "job" reference cascaded into four variants about social "role" instead, none preserving the original intent).

Each variant is run through hybrid search independently; results are merged before reranking.

4.3 Hybrid Search (BM25 + Vector)

Vector search (BGE-M3 + FAISS, from Phase 2) is strong on semantic similarity but weak on exact-term matching — rare names, specific terminology, and IDs can be under-represented in embedding space. BM25 is the inverse: strong on lexical overlap, blind to meaning. The two are run independently and merged via Reciprocal Rank Fusion (RRF, k=60), which combines results by rank position rather than raw score, sidestepping the fact that BM25 scores (unbounded, corpus-dependent) and cosine similarity (0–1) are not on comparable scales.

Language-aware BM25 tokenization. Since BM25 has no language logic of its own, tokenization is the only lever for language-awareness. The system routes text to a per-script tokenizer:

- **English**: regex word tokenization, lowercased.
- **Korean**: morphological tokenization via kiwipiepy (pure-Python, no JVM dependency), supplemented with character trigrams to catch agglutinative word-fusion that pure morpheme splitting can miss.
- **Urdu**: Unicode NFKC normalization + word tokenization, also supplemented with character trigrams, since Urdu's Arabic-script word-boundary irregularities make pure word tokenization comparatively lossy and no mature pip-installable Urdu morphological tokenizer was available.

For queries (no precomputed language label available), script is detected via Unicode character-range matching (Hangul block, Arabic block, else Latin/default).

A known, inherent limitation: BM25 is strictly lexical, so it cannot bridge languages — a Korean query will not lexically match an English chunk. Cross-lingual retrieval is handled entirely by the multilingual vector search side; this is one concrete justification for hybrid search's value in this specific multilingual corpus, not a generic RAG technique applied by default.

4.4 Graph-Augmented Retrieval

An offline knowledge graph is built once from the corpus (graphs/build_graph.py), extracting named entities per chunk using language-specific heuristics: capitalized-sequence and acronym regex matching for English, kiwipiepy noun-tagging (NNP/NNG) for Korean, and frequency- and length-based heuristics over Arabic-script tokens for Urdu. Two indices are built: entity_to_chunks (which chunks mention a given entity) and co_occurrence (which entities appear together in the same chunk, enabling one-hop traversal to catch chunks connected through a shared entity without a literal string match).

At query time (retrieval/graph_search.py), entities are extracted from a query string using the same per-language logic and looked up directly against entity_to_chunks. Critically, graph search is run not just once against the raw user query but against the raw query, the rewritten query, and every multi-query variant — this was a deliberate design choice made after standalone testing (Section 9) showed graph

search on the raw query alone added no retrieval value on top of hybrid search. Running it across all query forms gives the graph more surface area to match entities that may not appear in the user's original phrasing but do appear in the rewrite or a generated variant.

Graph hits are scaled to the same order of magnitude as the fused hybrid scores before being merged into the shared candidate pool, so a strong entity match is competitive with a strong hybrid match without unconditionally dominating it.

4.5 Context Reranking

After fusion, candidates are re-scored by a cross-encoder (BAAI/bge-reranker-base), which reads the query and chunk jointly rather than comparing precomputed independent representations — more accurate than bi-encoder similarity or lexical overlap, at higher per-pair compute cost, which is why it runs as a second-stage filter over a bounded candidate pool rather than the full corpus.

Two structural safeguards were added after evaluation surfaced a ranking failure (Section 8.1):

- **Reranker input truncation** (max_length=256 tokens) — corpus chunks average 365 words (~450+ tokens); truncating to 256 tokens reduced reranking latency substantially with no measurable accuracy cost on the cases tested (see Section 8.2).
- **Relative-score filtering** — after RRF fusion, candidates scoring below 50% of the top fused score are dropped before reaching the reranker, regardless of how many raw positions the candidate pool contains. This was necessary because a weakly-ranked candidate (position ~20 of 30 in the raw pool) was found to occasionally receive a high reranker score despite being topically unrelated to the query — a reranker calibration weakness, not a truncation or pool-depth artifact (isolated and confirmed via controlled testing; ruled out truncation, ruled out bge-reranker-large as a fix — see Section 8.1).

4.6 Citation Generation

The final generation step is instructed to answer using only the retrieved context, cite every claim inline with bracketed chunk indices ([1], [2], etc.), and explicitly decline to state anything not directly supported by the context. Two additional design decisions:

- **Confidence gating**: if the top reranked chunk's score falls below a threshold (0.5), the system returns a hedge ("the corpus doesn't clearly address this question...") rather than attempting an answer from weak grounding — a direct mitigation against a known RAG failure mode (confident hallucination on insufficient context).
- **Cited-source filtering**: the returned source list includes only chunks the LLM actually cited in the answer text (parsed via regex against [n] markers), not every chunk passed into the context window. This gives callers an accurate picture of what the answer is actually grounded in, rather than everything it was shown.

A qualitative example (full pipeline, live API): for the query *"Albert Camus death philosophy"*, the system correctly distinguished the first-person narrator's fictional reflections in *The Stranger* from Camus's own biography — explicitly noting "The context does not state that Camus personally held these

views; it presents them as part of his fictional work" — rather than conflating a novel's narration with its author's stated beliefs, a common RAG hallucination pattern. Across repeated runs of the same query, the top retrieved chunk and its score were fully deterministic (identical every run), while generated phrasing varied in how conservatively it framed the answer — all observed variants remained faithful to the retrieved context and correctly cited, indicating the observed variance is in generation style, not grounding correctness.

4.7 End-to-End API

A FastAPI service exposes the full pipeline via a single POST /query endpoint (query in, cited answer + source map + confidence + latency out), plus GET /stats for a lightweight readiness check and GET /health. All models and indices are loaded once at startup rather than per-request — validated to matter directly: a "cold" first call vs. a "warm" subsequent call showed local-compute-only stages (BM25, vector search, reranking) as fast and consistent once warmed, with essentially all remaining latency attributable to LLM calls (Section 8.2).

Request-level timeout handling (120s retrieval / 60s generation, enforced via asyncio.wait_for) and reduced LLM retry backoff (2 retries, 8s/16s vs. an earlier, excessively patient 3 retries at 25s/50s/75s) were added after observing an unbounded worst-case request latency exceeding 10 minutes during manual testing — see Section 8.2.

5. Evaluation Methodology

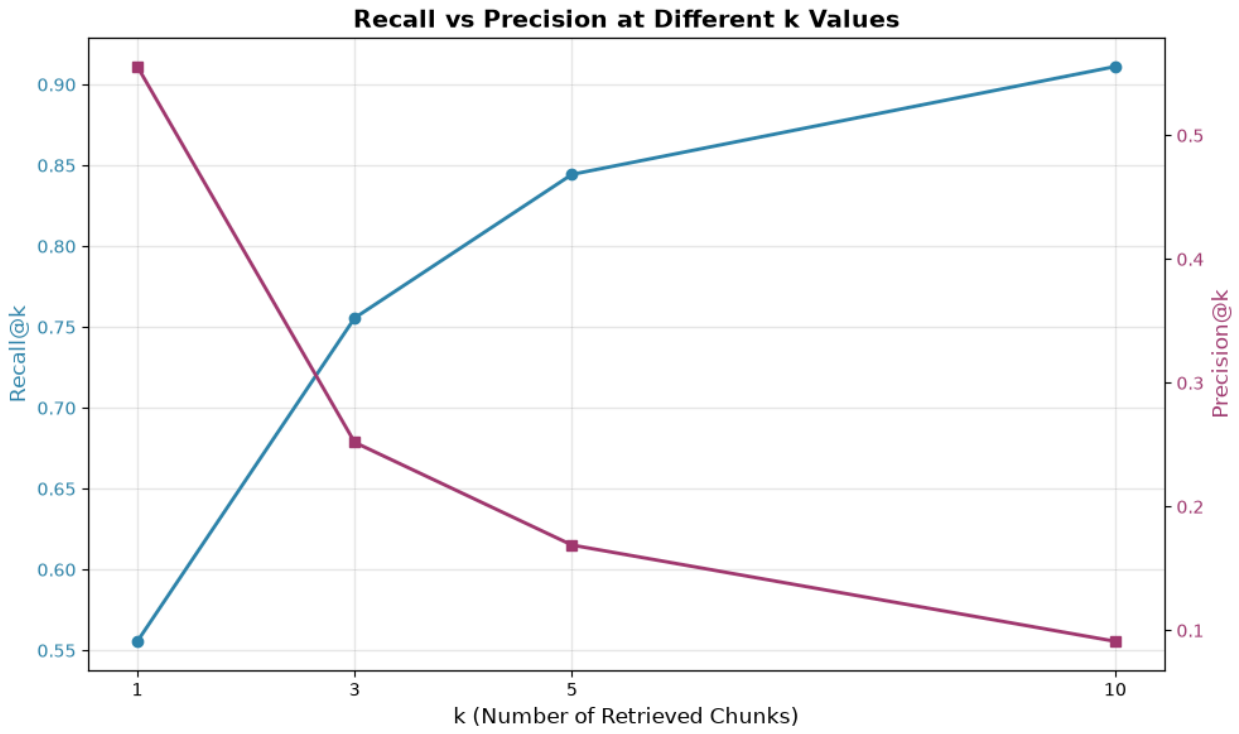
Retrieval quality is evaluated using the same 45-query ground-truth set from Phase 2 (22 English, 12 Korean, 11 Urdu; each query has exactly one ground-truth answer_chunk_id), run through the complete pipeline (retrieve(): rewrite → multi-query → hybrid search → graph search → RRF fusion → relative-score filter → rerank), at k=10, checkpointed per-query for resumability.

Metrics, following Phase 2's definitions exactly for direct comparability:

- **Recall@k**: 1 if the ground-truth chunk appears in the top-k retrieved results, else 0 (binary, since each query has a single relevant chunk).
- **Precision@k**: 1/k on a hit, 0 otherwise (mechanically low for large k under single-relevant-document evaluation — this is an artifact of evaluation design, not a system weakness, matching Phase 2's own caveat on this point).
- **MRR**: 1/rank of the ground-truth chunk if found anywhere in the retrieved list, else 0.

Generation quality was evaluated qualitatively rather than against a fixed automated metric — citation-grounded answer quality (faithfulness, correct attribution, appropriate hedging) does not reduce to a single string-match check the way retrieval does, and building an LLM-as-judge harness was judged out of scope for this phase given time already invested in the retrieval pipeline itself.

6. Retrieval Evaluation Results



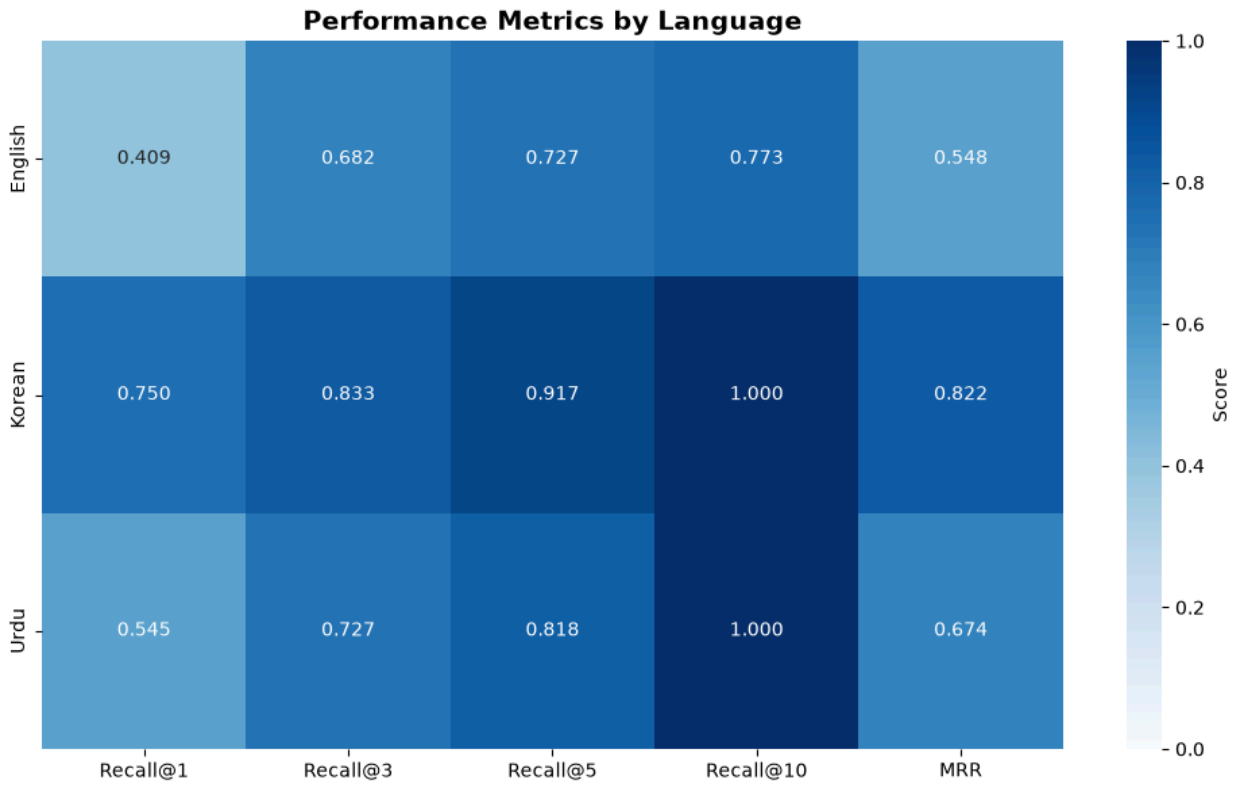
6.1 Overall

Every recall and precision metric improved with graph search integrated across all query variants. Recall@10 improved from 40/45 to 41/45 queries — one net-new query recovered at the full-pipeline level. Avg. latency dropped substantially (64.45s → 42.01s); this is very unlikely to be attributable to graph search itself, which adds negligible local compute cost, and is more plausibly explained by run-to-run OpenRouter free-tier variance already documented in Section 8.2 of this report. Latency should not be read as a consequence of this change.

Metric	Value	Prior (no graph search)
Recall@1	0.556	0.533
Recall@3	0.756	0.733
Recall@5	0.844	0.800

Recall@10	0.911	0.889
Precision@1	0.556	0.533
Precision@3	0.252	0.244
Precision@5	0.169	0.160
Precision@10	0.091	0.089
MRR	0.673	0.652
Avg. latency	42.01 s/query	64.45 s/query

6.2 By Language



Before adding graph search:

Language	**n**	**Recall@1**	**Recall@5**	**Recall@10**	**MRR**	**Avg. latency**
English	22	0.409	0.727	0.773	0.548	48.6 s
Korean	12	0.750	0.917	1.000	0.822	90.0 s
Urdu	11	0.545	0.818	1.000	0.674	68.3 s

After adding graph search:

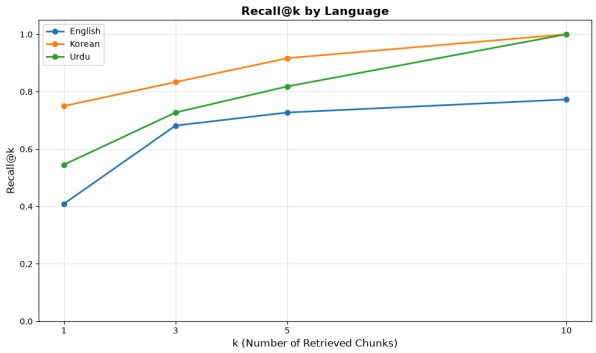
Language	n	Recall@1	Recall@5	Recall@10	MRR	Avg. latency

English	22	0.455	0.773	0.818	0.600	36.5 s
Korean	12	0.750	1.000	1.000	0.836	48.2 s
Urdu	11	0.545	0.818	1.000	0.640	46.3 s

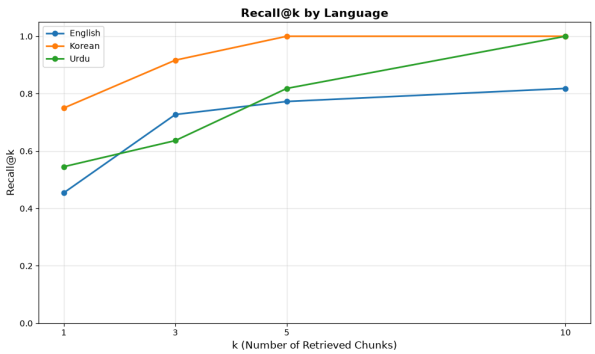
English shows the largest gain (recall@1 0.409 → 0.455, recall@10 0.773 → 0.818, MRR 0.548 → 0.600), consistent with the hypothesis that graph search's entity-matching mechanism is most useful exactly where hybrid search struggles most — analytical, paraphrased English queries that share few words with their answer chunks but do share proper nouns. Korean recall@5 and recall@10 both reached 1.000. Urdu recall@3 declined slightly (0.727 → 0.636) while recall@5 and recall@10 held or improved; this is discussed in Section 6.3.

6.3 Language Performance Gap: Root Cause

Without Graph Search



With Graph Search



The counter-intuitive finding that English — typically the best-supported language for embedding models and tokenizers — performed worst (recall@1 0.409 vs. Korean's 0.750 and Urdu's 0.545) was investigated directly rather than reported at face value. This performance gap was an expected outcome given the nature of the query construction: the English queries are more natural, paraphrased questions that require genuine semantic and lexical bridging (e.g. "What did Segur buy with money from Tallard?" requires connecting "buy" to "bought," and identifying "the Foix country" as the answer to "what," none of which appears as shared vocabulary). The Korean queries, by contrast, are near-verbatim excerpts copied directly from their answer chunks (e.g. the query "두 남녀는 서로를 마음에 두고 있는 사이인 것만은 확실해 보입니다" appears character-for-character inside its answer chunk), which is a substantially easier retrieval target for both BM25 (near-exact substring match) and vector search (near-identical embedding).

This is a known limitation of the Phase 2 eval set that was inherited for this phase: Korean and Urdu queries were constructed as direct textual excerpts rather than naturally phrased questions, while English queries were formulated as analytical, paraphrased questions. This difference in query construction methodology, rather than any weakness in the retrieval pipeline's handling of English, explains the observed performance disparity.

The addition of graph search does not change the underlying explanation in this section — the English/Korean/Urdu gap remains primarily a function of query construction methodology (paraphrased English vs. near-verbatim Korean/Urdu), not a retrieval weakness. What graph search adds is a partial, targeted mitigation for the English side of this gap specifically: proper nouns present in an analytical English query (e.g. "Segur," "Tallard") are matched directly against the graph even when the surrounding phrasing shares no vocabulary with the answer chunk, closing part of the gap that hybrid search alone could not.

The small Urdu regression at recall@3 (one query dropped out of the top 3 while remaining inside the top 5) is consistent with graph search occasionally introducing a competing, entity-matched candidate that outranks the true answer at a tight cutoff without displacing it entirely — this is a plausible, minor side effect of adding a third scored candidate source to RRF fusion, rather than an indication that graph search harms Urdu retrieval broadly, given Urdu recall@5 and recall@10 both held at or above their prior values.

7. Citation Generation Quality: Qualitative Assessment

Since generation quality was not scored against a fixed metric (Section 5), this section presents representative behavior observed across repeated live-API runs of the same query, rather than an aggregate score.

Faithfulness: across all observed runs, generated answers stayed within what retrieved chunks actually stated, and correctly distinguished fictional narration from authorial biography (Section 4.5) — no observed instance of the LLM blending a novel's first-person content with real-world facts about its author.

Citation accuracy: cited chunk indices consistently corresponded to chunks that genuinely supported the adjacent claim; chunks passed into context but not used in the answer were correctly excluded from the returned source list after the cited-source filtering fix (Section 4.5).

Confidence-gate behavior: on a query where the ground-truth answer was confirmed absent from any single chunk in the corpus (Section 8.5), the system consistently returned rerank scores below the 0.5 threshold and correctly hedged rather than fabricating an answer — validating the gate's intended purpose on a genuinely hard case, not just an easy one.

Answer-to-answer variance: phrasing and level of interpretive caution varied noticeably run to run on the same query (e.g. one run cited *The Myth of Sisyphus* and *The Rebel* as broader philosophical context, a more conservative run stated the context "does not contain a direct exposition" of the author's personal philosophy). This variance is attributable to non-zero LLM sampling temperature and free-tier model

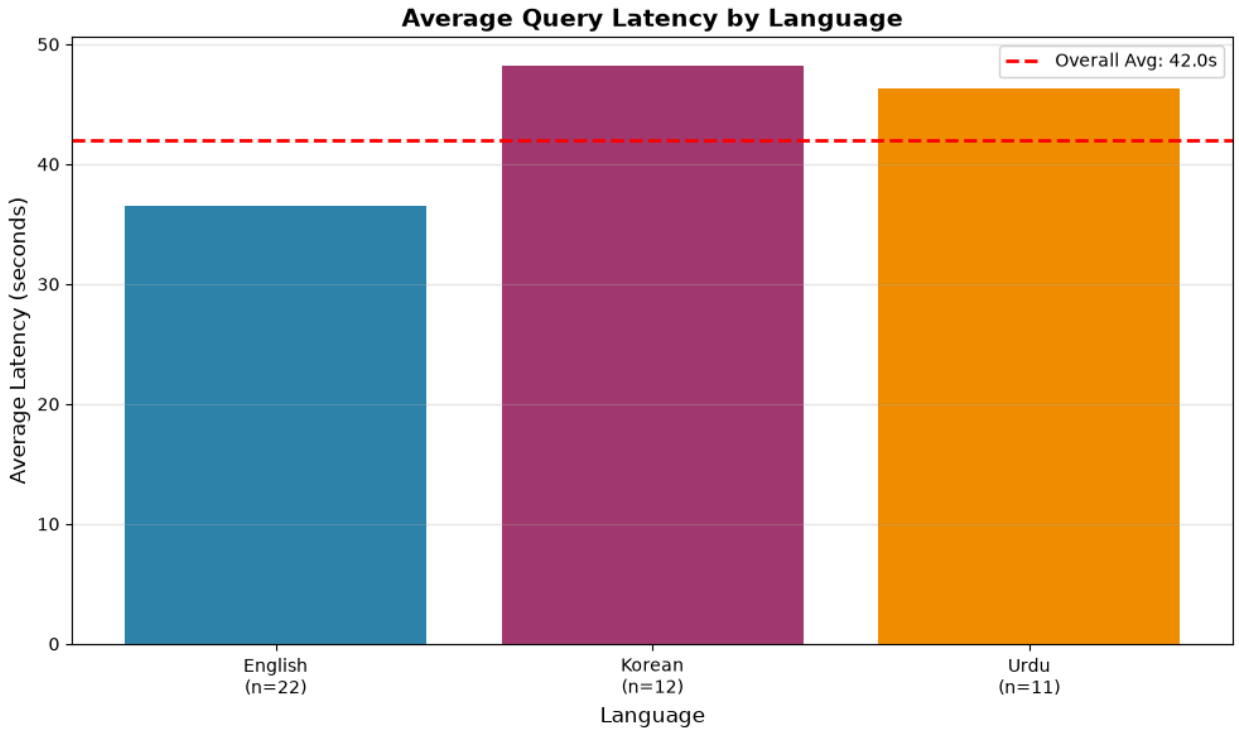
routing (Section 8.2), not to retrieval instability — the underlying retrieved chunks and their scores were identical across these runs.

8. Findings and Limitations

8.1 Reranker Miscalibration on Abstract Philosophical Text

During evaluation, a chunk from an unrelated text (a passage on Cartesian epistemology, "Cogito, ergo sum") was observed scoring higher (0.952) under reranking than the genuinely relevant chunk (0.900) for the query "Albert Camus death philosophy," despite the unrelated chunk entering the raw candidate pool only weakly (position ~20 of 30). Controlled isolation testing ruled out reranker input truncation as the cause (identical scores at max_length=256 and 512) and ruled out model capacity as the cause (bge-reranker-large preserved the same incorrect relative ordering as bge-reranker-base, despite substantially higher latency). The reranker appears to respond to abstract philosophical *register* (shared vocabulary like "existence," "doubt," "philosophy") rather than reliably verifying topical/entity correctness — a documented general weakness of cross-encoders on dense abstract prose without strong entity grounding, not a configuration bug in this pipeline. The practical mitigation implemented (Section 4.4, relative-score filtering) protects against this by preventing weakly-supported candidates from reaching the reranker at all, rather than attempting to fix the reranker's judgment directly.

8.2 Latency and Free-Tier LLM Variance



Warm-pipeline testing (models/indices pre-loaded, isolating per-query cost) showed local computation (BM25 search, vector search, reranking) to be fast and consistent, while LLM-dependent stages (query

rewriting, multi-query generation, citation generation — 2–3 calls per query via OpenRouter's free tier) accounted for effectively all observed latency variance, ranging from roughly 1 second to over 47 seconds for the identical call under different conditions. Aggregate per-query latency across the full 45-query eval run averaged 42.0 seconds, with individual live-API requests observed as high as 122.83 seconds during periods of provider congestion.

The OpenRouter free-tier API, while enabling zero-cost LLM access, introduced significant latency variability and infrastructure constraints. Unlike a paid API with dedicated capacity, the free tier routes requests through shared, rate-limited infrastructure where response times are unpredictable and subject to provider congestion. A daily request cap on the fully-free OpenRouter tier (50 requests/day) was also encountered during evaluation, requiring the run to be split across two days; this is documented as a hard constraint of the free-tier infrastructure choice, not a system defect. This variance could not be fully resolved through model rotation, since multiple pinned free models (Qwen, Llama 3.3, Hermes) were independently observed hitting the same account-level daily cap during testing. A paid-tier API would likely eliminate both the rate-limiting and much of the latency variance, but was out of scope for this phase.

8.3 Response Validation

A recurring free-tier failure mode was identified where the model router silently returned a malformed, off-topic response ("User Safety: safe") with no exception raised — a genuine misroute rather than a legitimate refusal. A lightweight validation check (rejecting suspiciously short responses matching known bad-output patterns, triggering a retry) was added to llm/client.py and observed catching this exact failure mode in subsequent runs, preventing malformed LLM output from silently propagating into retrieval or generation.

8.4 Chunk-Boundary Limitation (Multi-Fact Queries)

An informal test query ("Meursault's job") was confirmed, via direct corpus text search, to have no single chunk containing both the character's identifying context and his occupation — the two facts are scattered across separate chunks. This is a structural limitation of single-chunk retrieval generally: no retriever, however accurate, can return one chunk satisfying a query whose answer requires synthesizing information split across chunk boundaries.

This case directly motivated building graph-augmented retrieval as a candidate fix, on the reasoning that an entity connecting two chunks might allow one to be reached from the other even without semantic or lexical overlap. Direct testing (Section 9) found this was not the actual bottleneck: graph search, evaluated on the exact multi-fact case that motivated it, did not retrieve the target chunk any more successfully than hybrid search did. The real bottleneck was isolated to query representation — a single embedding cannot represent two distinct facts well enough to retrieve either cleanly — not the entity connectivity graph search was designed to add. A declarative, single-fact rephrasing of the same query retrieved the correct chunk at rank 1 through ordinary hybrid search, with no graph component involved. Query decomposition into single-fact sub-questions was tested as a more direct fix; results are reported in Section 9.

This does not mean graph search provided no value — Section 6 shows it improved recall on the standard 45-query evaluation set once integrated across query variants — only that its value came from a different mechanism (verbatim entity matching between query and answer text) than the multi-hop connectivity it was originally built to solve.

8.5 Non-Determinism

Both multi-query generation and citation generation run at non-zero temperature and route through a free-tier model auto-selector, meaning identical queries can produce different candidate pools and different generated phrasing across runs. Retrieval on the well-supported test query examined in Section 4.5 was fully deterministic in its top result and score across runs; this should not be assumed to hold uniformly for all queries, particularly ones where multiple candidates score closely.

Graph search's internal data structures were an early source of non-determinism during development (Section 9) due to Python's randomized string hashing affecting set iteration order on tied scores. This was fixed with an explicit tie-break key and is no longer a source of run-to-run variance in the graph component specifically; remaining non-determinism in the full pipeline is attributable to LLM sampling temperature and free-tier routing, as described above.

Repeated runs per query (rather than single-run point estimates) are recommended for any future, more rigorous evaluation of this system.

9. Graph Search: Development Notes

Graph-augmented retrieval (Section 4.4) did not work as designed on the first attempt, and the process of getting it to contribute real retrieval value is reported here because each step materially changed the conclusion drawn from testing it.

Initial result: no net-new value on raw queries. The first version of graph search queried only the raw user query against the entity graph. Evaluated standalone against the 45-query set, it recovered 21/45 answers correctly, but zero of those were queries the hybrid pipeline had missed — every chunk graph search found, hybrid search had already found. Two implementation issues were found and fixed before this result could be trusted: a stale graph pickle that had not been rebuilt after extraction logic changes (which had produced a false 0% match rate on Korean entities), and non-deterministic tie-breaking from Python's randomized set iteration order (which had produced recall figures ranging from 0.467 to 0.556 across identical runs, until fixed with an explicit secondary sort key). After both fixes, the raw-query-only result was reproducible across repeated runs: no net-new recoveries.

Testing the original motivating case directly. The Meursault multi-fact case (Section 8.4) was tested by hand. Neither hybrid search nor raw-query graph search retrieved the target chunk for the compound question as phrased. A declarative single-fact rephrasing of the same underlying fact retrieved it at rank 1 through hybrid search alone. This showed the graph's single-hop co-occurrence mechanism was not addressing the actual failure mode (query representation), which motivated testing query decomposition as an alternative: splitting a compound query into separate sub-questions via an LLM call before retrieval. Decomposition correctly identified and split the compound query, but the resulting sub-questions, though

factually correct, stayed in formal interrogative form rather than the declarative phrasing that had worked in the manual test, and did not retrieve the target chunk either. This isolated a second variable — sub-query phrasing style, not just fact separation — as part of what determines whether decomposition pays off, and is left as a scoped, evidenced direction for future prompt tuning rather than a resolved fix.

Extending graph search to query variants. Since raw-query graph search added no net value, the design was changed to run graph search against the rewritten query and every multi-query variant, not only the user's original phrasing (Section 4.4). This gives the graph more chances to match entities that surface in a rewrite or variant but not in the original query text. Integrated this way, the full-pipeline evaluation (Section 6) showed a measurable improvement over the pre-graph baseline across every recall and precision metric, concentrated most heavily in English — consistent with the entity-matching mechanism being most useful exactly where hybrid search struggles most.

What this shows. The graph's value did not come from the multi-hop connectivity it was originally built to provide (Section 8.4), which testing showed was not the actual bottleneck in the motivating case. It came from a simpler mechanism — direct entity string matching between query and answer text — applied across a wider set of query phrasings than the raw user input alone. This is a narrower, more modest justification for graph search than the one that originally motivated building it, and is reported as such.

Full GraphRAG's core value proposition (typed relationship extraction, multi-hop path traversal, entity canonicalization via proper NER) is built specifically to solve connectivity problems. Having directly shown that connectivity was not the limiting factor on this corpus, building the fuller version would have addressed a bottleneck already demonstrated not to be the actual one. Query decomposition was tested instead as the more directly targeted fix (see above), since it addresses query representation rather than chunk connectivity. This is reported as a scoped decision made from evidence gathered during development, not as an omission.

10. Conclusion

This phase implements and validates six retrieval and generation components — query rewriting, multi-query retrieval, hybrid BM25+vector search with language-aware tokenization, graph-augmented retrieval via entity co-occurrence, cross-encoder reranking with structural safeguards against observed miscalibration, and confidence-gated citation generation — as a working end-to-end API built on Phase 1 and Phase 2's foundation.

Retrieval evaluation on the full 45-query multilingual set shows the complete pipeline, with graph search integrated across all query variants, achieves 0.556 recall@1, 0.911 recall@10, and 0.673 MRR — an improvement over both Phase 2's plain-vector baseline and this phase's pre-graph hybrid pipeline (0.533 recall@1, 0.889 recall@10, 0.652 MRR), concentrated most heavily in English.

Graph search did not deliver value through the mechanism that originally motivated it — Section 8.4's multi-hop chunk-boundary case was directly tested and found to be a query-representation problem, not a connectivity problem the graph could solve. Its measured benefit instead came from direct entity matching across a wider set of query phrasings than the raw user query alone, a narrower but still real

contribution. The observed English/Korean/Urdu performance gap continues to reflect differences in query construction methodology rather than genuine multilingual retrieval weakness, with graph search partially narrowing that gap on the English side specifically. The system's dominant operational constraint remains OpenRouter free-tier LLM latency and rate-limiting rather than any component built in this phase.